

UNIT I

Introduction to Software Engineering and Software Process Models

Software Engineering:

- Software engineering encompasses a process, a collection of methods (practice) and an array of tools that allow professionals to build high-quality computer software.
- Software engineering is important because it enables us to build complex systems in a timely manner and with high quality.
- Software engineers build and support software, and virtually everyone in the industrialized world uses it either directly or indirectly.
- Software is important because it affects nearly every aspect of our lives and has become pervasive in our commerce, our culture, and our everyday activities. Software engineering is important because it enables us to build complex systems in a timely manner and with high quality.
- You build computer software like you build any successful product, by applying an agile, adaptable process that leads to a high-quality result that meets the needs of the people who will use the product. You apply a software engineering approach.
- From the point of view of a software engineer, the work product is the set of programs, content (data), and other work products that are computer software. But from the user's viewpoint, the work product is the resultant information that somehow makes the user's world better. How do I ensure that I've done it right?

Software Engineering Fundamentals :

Nature of Software :

- Today, software takes on a dual role. It is a product, and at the same time, the vehicle for delivering a product.
- As a product, it delivers the computing potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware.
- Whether it resides within a mobile phone or operates inside a mainframe computer, software is an information transformer producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as a single bit or as complex as a multimedia presentation derived from data acquired from dozens of independent sources.
- As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).
- Software delivers the most important product of our time information.

- It transforms personal data (e.g., an individual's financial transactions) so that the data can be more useful in a local context; it manages business information to enhance competitiveness; it provides a gateway to worldwide information networks (e.g., the Internet), and provides the means for acquiring information in all of its forms.

Defining Software:

Software is:

- (1) instructions (computer programs) that when executed, it provide desired features, function, and performance.
- (2) data structures that enable the programs to adequately manipulate information, and
- (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

Therefore, software has characteristics that are considerably different than those of hardware :-

1. Software is developed or engineered. it is not manufactured in the classical sense.
 - Although some similarities exist between software development and hardware manufacturing, the two activities are fundamentally different.
 - In both activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems that are nonexistent (or easily corrected) for software.
 - Both activities are dependent on people, but the relationship between people applied and work accomplished is entirely different.
 - Both activities require the construction of a "product," but the approaches are different.
 - Software costs are concentrated in engineering. This means that software projects cannot be managed as if they were manufacturing projects

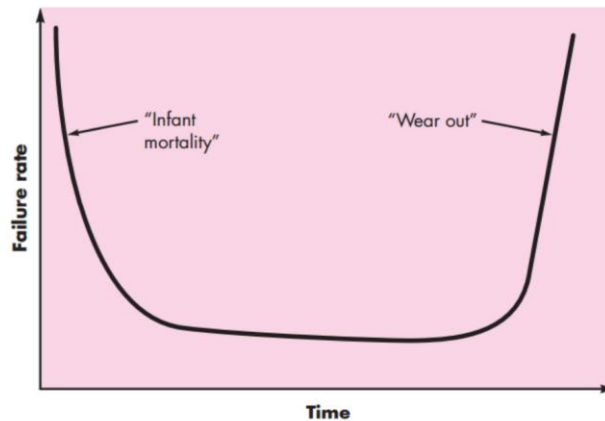


Figure 1.1 :- Failure curve for Hardware

- Figure 1.1 depicts failure rate as a function of time for hardware.
- The relationship, often called the “bathtub curve,” indicates that hardware exhibits relatively high failure rates early in its life (these failures are often attributable to design or manufacturing defects). defects are corrected and the failure rate drops to a steady-state level (hopefully, quite low) for some period of time.
- As time passes, however, the failure rate rises again as hardware components suffer from the cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies.
- Stated simply, the hardware begins to wear out

2. Software doesn’t “wear out.”

- Software is not susceptible to the environmental maladies that cause hardware to wear out.
- In theory, therefore, the failure rate curve for software should take the form of the “idealized curve” shown in Figure 1.2

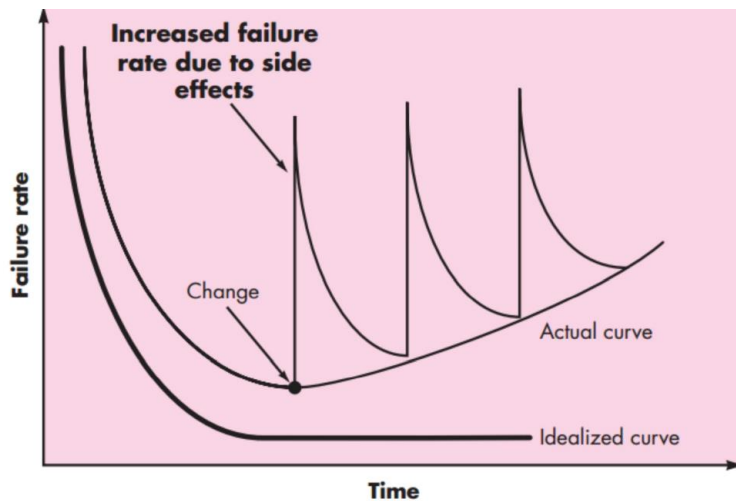


Figure 1.2 :- Failure curves for Software

- The idealized curve is a gross oversimplification of actual failure models for software.
- However, the implication is clear—software doesn't wear out. But it does deteriorate!
- During its life, software will undergo change. → As changes are made, it is likely that errors will be introduced, causing the failure rate curve to spike as shown in the “actual curve” (Figure 1.2).
- Before the curve can return to the original steady-state failure rate, another change is requested, causing the curve to spike again. Slowly, the minimum failure rate level begins to rise—the software is deteriorating due to change.
- When a hardware component wears out, it is replaced by a spare part.
- There are no software spare parts.
- Every software failure indicates an error in design or in the process through which design was translated into machine executable code.
- Therefore, the software maintenance tasks that accommodate requests for change involve considerably more complexity than hardware maintenance.
 - Although the industry is moving toward component-based construction, most software continues to be custom built. As an engineering discipline evolves, a collection of standard design components is created.
 - Standard screws and off-the-shelf integrated circuits are only two of thousands of standard components that are used by mechanical and electrical engineers as they design new systems.
 - The reusable components have been created so that the engineer can concentrate on the truly innovative elements of a design, that is, the parts of the design that represent something new.
 - In the hardware world, component reuse is a natural part of the engineering process.

- In the software world, it is something that has only begun to be achieved on a broad scale.
- A software component should be designed and implemented so that it can be reused in many different programs.
- For example, today's interactive user interfaces are built with reusable components that enable the creation of graphics windows, pull-down menus, and a wide variety of interaction mechanisms.
- The data structures and processing detail required to build the interface are contained within a library of reusable components for interface construction.

Software Engineering Practice:

introduced a generic software process model composed of a set of activities that establish a framework for software engineering practice. Generic framework activities—communication, planning, modeling, construction, and deployment—and umbrella activities establish a skeleton architecture for software engineering work. But how does the practice of software engineering fit in? In the sections that follow, you'll gain a basic understanding of the generic concepts and principles that apply to framework activities.

In the sections that follow, you'll gain a basic understanding of the generic concepts and principles that apply to framework activities

The Essence of Practice

1. Understand the problem (communication and analysis).
2. Plan a solution (modeling and software design).
3. Carry out the plan (code generation).
4. Examine the result for accuracy (testing and quality assurance).

1. Understand the problem (communication and analysis).

It's sometimes difficult to admit, but most of us suffer from hubris (presumption) when we're presented with a problem. We listen for a few seconds and then think, Oh yeah, I understand, let's get on with solving this thing.

Unfortunately, understanding isn't always that easy. It's worth spending a little time answering a few simple questions:

Who has a stake in the solution to the problem? That is, who are the stakeholders?

What are the unknowns? What data, functions, and features are required to properly solve the problem? Can the problem be compartmentalized? Is it possible to represent smaller problems that may be easier to understand?

Can the problem be represented graphically? Can an analysis model be created?

2. Plan the Solution

Now you understand the problem (or so you think) and you can't wait to begin coding. Before you do, slow down just a bit and do a little design.

Have you seen similar problems before? Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?

Has a similar problem been solved? If so, are elements of the solution reusable?

Can sub problems be defined? If so, are solutions readily apparent for the sub problems?

Can you represent a solution in a manner that leads to effective implementation? Can a design model be created?

3. Carry out the Plan

The design you've created serves as a road map for the system you want to build. There may be unexpected detours, and it's possible that you'll discover an even better route as you go, but the "plan" will allow you to proceed without getting lost.

Does the solution conform to the plan? Is source code traceable to the design model?

Is each component part of the solution provably correct? Have the design and code been reviewed, or better, have correctness proofs been applied to the algorithm?

4. Examine the Result

You can't be sure that your solution is perfect, but you can be sure that you've designed a sufficient number of tests to uncover as many errors as possible. ☞ Is it possible to test each component part of the solution? Has a reasonable testing strategy been implemented?

Does the solution produce results that conform to the data, functions, and features that are required? Has the software been validated against all stakeholder requirements? —

In fact, it's reasonable to state that a commonsense approach to software engineering will never lead you astray (away from the right path)

The Software Process :

It is a set of activities, methods, practices, and transformations that people use to develop and maintain software and its associated products. As a Software Organization, the software process becomes better defined and more consistently implemented throughout the organization.

Example: Project plans, Design-Documents, Code, Test cases, and User-manuals

Generic Process Model:

A process was defined as a collection of work activities, actions, and tasks that are performed when some work product is to be created. Each of these activities, actions, and tasks reside within a framework or model that defines their relationship with the process and with one another.

each framework activity is populated by a set of software engineering actions. Each software engineering action is defined by a task set that identifies the work tasks that are to be completed, the work products that will be produced, the quality assurance points that will be required, and the milestones that will be used to indicate progress.

The Generic process model is an abstraction of the [software development process](#). It specifies the stages and order of a process. The generic Process Model will define the following:

- i. The tasks to be performed.
- ii. The input and output of each task.
- iii. The pre and post-conditions for each task.
- iv. The flow and sequence of each task.

Generic Process Framework Activities:

It establishes the foundation for a complete software process by identifying a small number of framework activities. It also includes a set of umbrella activities that are applicable across the entire software process. Each framework activity is populated by a set of software engineering actions. A Generic Process Framework for Software Engineering encompasses five activities

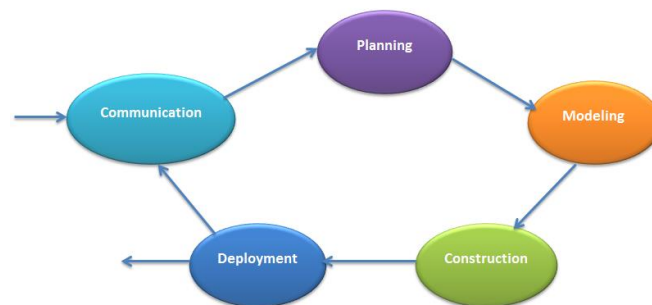
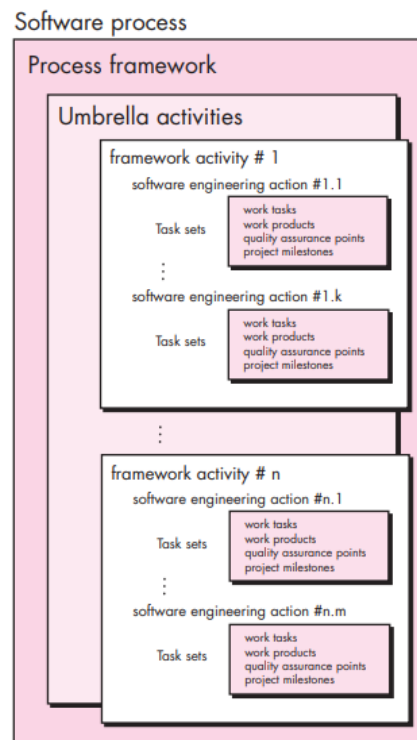


Fig: Generic Process Model

1. **Communication:** This framework activity involves heavy communication and collaboration with the customer. It encompasses requirements gathering and other related activities.
2. **Planning:** This activity establishes a plan for the software engineering work that follows. It describes the technical tasks which are conducted. The resource requires and the work products produced with a work schedule.
3. **Modeling:** It encompasses the creation of models that allow the developer and the customer. It is easy to better understand Software requirements and the design that will achieve those requirements.
4. **Construction:** This activity combines code generation and the testing that is required to uncover errors in the code.
5. **Deployment:** The software delivered to the customer who evaluates the delivered product. It also provides feedback based on the evaluation.

Defining a Framework Activity

Although I have described five framework activities and provided a basic definition a software team would need significantly more information before it could properly execute any one of these activities as part of the software process. Therefore, you are faced with a key question: What actions are appropriate for a framework activity, given the nature of the problem to be solved, the characteristics of the people doing the work, and the stakeholders who are sponsoring the project?



Identifying a Task Set

each software engineering action (e.g., elicitation, an action associated with the communication activity) can be represented by a number of different task sets—each a collection of software engineering work tasks, related work products, quality assurance points, and project milestones. You should choose a task set that best accommodates the needs of the project and the characteristics of your team. This implies that a software engineering action can be adapted to the specific needs of the software project and the characteristics of the project team.

Software Myths:

Software myths—erroneous beliefs about software and the process that is used to build it—can be traced to the earliest days of computing. Myths have a number of attributes that make them insidious. For instance, they appear to be reasonable statements of fact (sometimes containing elements of truth), they have an intuitive feel, and they are often promulgated by experienced practitioners who “know the score.”

1. Management myths :

Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality.

- Myth : We already have a book that’s full of standards and procedures for building software. Won’t that provide my people with everything they need to know?
- Reality : The book of standards may very well exist, but is it used? Are software practitioners aware of its existence? Does it reflect modern software engineering practice? Is it complete? Is it adaptable? Is it streamlined to improve time-to-delivery while still maintaining a focus on quality? In many cases, the answer to all of these questions is “no.”
- Myth : If we get behind schedule, we can add more programmers and catch up (sometimes called the “Mongolian horde” concept).
- Reality : Software development is not a mechanistic process like manufacturing. People can be added but only in a planned and well coordinated manner.
- Myth : If I decide to outsource the software project to a third party, I can just relax and let that firm build it.
- Reality : If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it out-sources software projects.

2. Customer myths

A customer who requests computer software may be a person at the next desk, a technical group down the hall, the marketing/sales department, or an outside company that has requested software under contract. Myths lead to false expectations (by the customer) and, ultimately, dissatisfaction with the developer.

- Myth : A general statement of objectives is sufficient to begin writing programs—we can fill in the details later.
- Reality : Although a comprehensive and stable statement of requirements is not always possible, an ambiguous “statement of objectives” is a recipe for disaster. Unambiguous requirements (usually derived iteratively) are developed only through effective and continuous communication between customer and developer.

- Myth : Software requirements continually change, but change can be easily accommodated because software is flexible.
- Reality : It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early (before design or code has been started), the cost impact is relatively small.

3. Practitioner's myths

Myths that are still believed by software practitioners have been fostered by over 50 years of programming culture. During the early days, programming was viewed as an art form. Old ways and attitudes die hard.

- Myth : Once we write the program and get it to work, our job is done.
- Reality : Someone once said that "the sooner you begin 'writing code,' the longer it'll take you to get done." Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.
- Myth : Until I get the program "running" I have no way of assessing its quality.
- Reality : One of the most effective software quality assurance mechanisms can be applied from the inception of a project the technical review. Software reviews are a "quality filter" that have been found to be more effective than testing for finding certain classes of software defects.
- Myth : The only deliverable work product for a successful project is the working program.
- Reality : A working program is only one part of a software configuration that includes many elements. A variety of work products (e.g., models, documents, plans) provide a foundation for successful engineering and, more important, guidance for software support.
- Myth : Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.
- Reality : Software engineering is not about creating documents. It is about creating a quality product. Better quality leads to reduced rework. And reduced rework results in faster delivery times.

Process Assessment and Improvement:

The existence of a software process is no guarantee that software will be delivered on time, that it will meet the customer's needs, or that it will exhibit the technical characteristics that will lead to long-term quality characteristics .

Process patterns must be coupled with solid software engineering practice . In addition, the process itself can be assessed to ensure that it meets a set of basic process criteria that have been shown to be essential for a successful software engineering.

A number of different approaches to software process assessment and improvement have been proposed over the past few decades:

Standard CMMI Assessment Method for Process Improvement (SCAMPI)—

provides a five-step process assessment model that incorporates five phases: initiating, diagnosing, establishing, acting, and learning. The SCAMPI method uses the SEI CMMI as the basis for assessment.

CMM-Based Appraisal for Internal Process Improvement (CBA IPI)—

provides a diagnostic technique for assessing the relative maturity of a software organization; uses the SEI CMM as the basis for the assessment [Dun01].

SPICE (ISO/IEC15504)—

a standard that defines a set of requirements for software process assessment. The intent of the standard is to assist organizations in developing an objective evaluation of the efficacy of any defined software process [ISO08].

ISO 9001:2000 for Software—

a generic standard that applies to any organization that wants to improve the overall quality of the products, systems, or services that it provides. Therefore, the standard is directly applicable to software organizations and companies [Ant06]. A more detailed discussion of software assessment and process improvement methods is presented.

Prescriptive Process Models:

Prescriptive process models were originally proposed to bring order to the chaos of software development. History has indicated that these traditional models have brought a certain amount of useful structure to software engineering work and have provided a reasonably effective road map for software teams. However, software engineering work and the product that it produces remain on “the edge of chaos.” In an intriguing paper on the strange relationship between order and chaos in the software world, Nogueira and his colleagues [Nog00] state.

The edge of chaos is defined as “a natural state between order and chaos, a grand compromise between structure and surprise” [Kau95]. The edge of chaos can be visualized as an unstable, partially structured state.

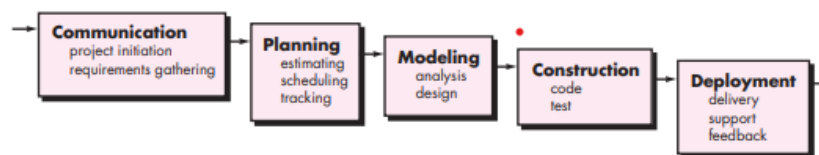
It is unstable because it is constantly attracted to chaos or to absolute order. We have the tendency to think that order is the ideal state of nature. This could be a mistake. Research . . . supports the theory that operation away from equilibrium generates creativity, self-organized processes, and increasing returns [Roo96]. Absolute order means the absence of variability, which could be an advantage under unpredictable environments. Change occurs when there is

some structure so that the change can be organized, but not so rigid that it cannot occur. Too much chaos, on the other hand, can make coordination and coherence impossible. Lack of structure does not always mean disorder.

The philosophical implications of this argument are significant for software engineering. If prescriptive process models⁵ strive for structure and order, are they inappropriate for a software world that thrives on change? Yet, if we reject traditional process models (and the order they imply) and replace them with something less structured, do we make it impossible to achieve coordination and coherence in software work?

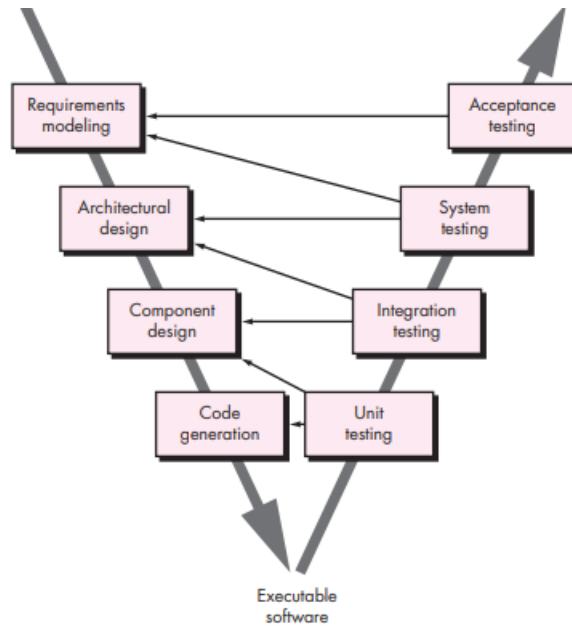
There are no easy answers to these questions, but there are alternatives available to software engineers. In the sections that follow, I examine the prescriptive process approach in which order and project consistency are dominant issues. I call them “prescriptive” because they prescribe a set of process elements—framework activities, software engineering actions, tasks, work products, quality assurance, and change control mechanisms for each project. Each process model also prescribes a process flow (also called a work flow)—that is, the manner in which the process elements are interrelated to one another.

The Waterfall Model:



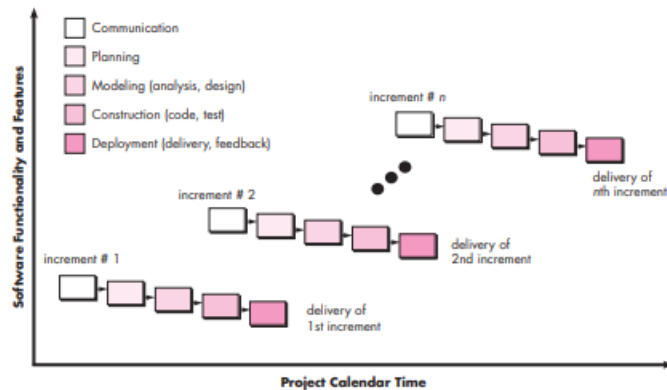
There are times when the requirements for a problem are well understood—when work flows from communication through deployment in a reasonably linear fashion. The waterfall model, sometimes called the classic life cycle, suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment, culminating in ongoing support of the completed software. A variation in the representation of the waterfall model is called the V-model.

The V-model



- As a software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution.
- Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions) that validate each of the models created as the team moved down the left side.
- In reality, there is no fundamental difference between the classic life cycle and the V-model. The V-model provides a way of visualizing how verification and validation actions are applied to earlier engineering work.
- The waterfall model is the oldest paradigm for software engineering. Among the problems that are sometimes encountered when the waterfall model is applied are:
 1. Real projects rarely follow the sequential flow that the model proposes. Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds.
 2. Real projects rarely follow the sequential flow that the model proposes. Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds.
 3. The customer must have patience. A working version of the program(s) will not be available until late in the project time span. A major blunder, if undetected until the working program is reviewed, can be disastrous.
- The linear nature of the classic life cycle leads to “blocking states” in which some project team members must wait for other members of the team to complete dependent tasks.
- In fact, the time spent waiting can exceed the time spent on productive work.

Incremental Process Models:

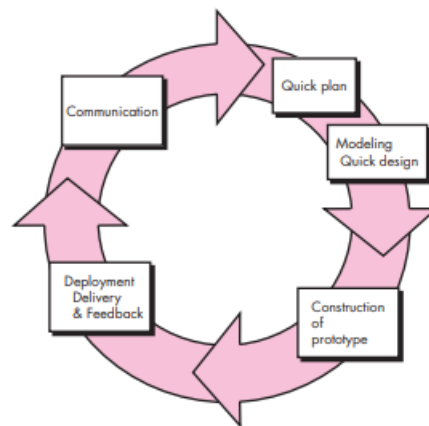


- The incremental model combines elements of linear and parallel process flow. For example, word-processing software developed using the incremental paradigm might deliver basic file management, editing, and document production functions in the first increment; more sophisticated editing and document production capabilities in the second increment; spelling and grammar checking in the third increment; and advanced page layout capability in the fourth increment.
- It should be noted that the process flow for any increment can incorporate the prototyping paradigm. When an incremental model is used, the first increment is often a core product. That is, basic requirements are addressed but many supplementary features (some known, others unknown) remain undelivered. The core product is used by the customer (or undergoes detailed evaluation). As a result of use and/or evaluation, plan is developed for the next increment. The plan addresses the modification of the core product to better meet the needs of the customer and the delivery of additional features and functionality.
- This process is repeated following the delivery of each increment, until the complete product is produced. The incremental process model focuses on the delivery of an operational product with each increment. Early increments are stripped-down versions of the final product, but they do provide capability that serves the user and also provide a platform for evaluation by the user.
- Early increments can be implemented with fewer people. If the core product is well received, then additional staff (if required) can be added to implement the next increment. In addition, increments can be planned to manage technical risks. For example, a major system might require the availability of new hardware that is under development and whose delivery date is uncertain.
- It might be possible to plan early increments in a way that avoids the use of this hardware, thereby enabling partial functionality to be delivered to end users without inordinate delay.

Evolutionary Process Models:

- Software, like all complex systems, evolves over a period of time. Business and product requirements often change as development proceeds. You need a process model that has been explicitly designed to accommodate a product that evolves over time. Prototyping Often, a customer defines a set of general objectives for software, but does not identify detailed requirements for functions and features.
- In other cases, the developer may be unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form that human machine interaction should take. In these, and many other situations, a prototyping paradigm may offer the best approach. The prototyping paradigm begins with communication.
- You meet with other stakeholders to define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory.
- A prototyping iteration is planned quickly, and modeling (in the form of a “quick design”) occurs. A quick design focuses on a representation of those aspects of the software that will be visible to end user.
- The quick design leads to the construction of a prototype. The prototype is deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements

FIGURE 2.6
The
prototyping
paradigm



- Ideally, the prototype serves as a mechanism for identifying software requirements. If a working prototype is to be built, you can make use of existing program fragments or apply tools (e.g., report generators and window managers) that enable working programs to be generated quickly.
- Both stakeholders and software engineers like the prototyping paradigm. Users get a feel for the actual system, and developers get to build something immediately. Yet, prototyping can be problematic for the following reasons:
 1. Stakeholders see what appears to be a working version of the software, unaware that the prototype is held together haphazardly, unaware that in the

rush to get it working you haven't considered overall software quality or long-term maintainability.

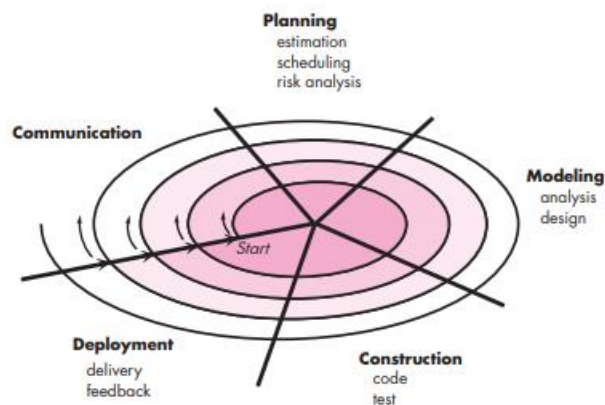
2. As a software engineer, you often make implementation compromises in order to get a prototype working quickly.

The Spiral Model:

- Originally proposed by Barry Boehm [Boe88], the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model. It provides the potential for rapid development of increasingly more complete versions of the software. Boehm [Boe01a] describes the model in the following manner:
- The spiral development model is a risk-driven process model generator that is used to guide multi-stakeholder concurrent engineering of software intensive systems. It has two main distinguishing features. One is a cyclic approach for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk. The other is a set of anchor point milestones for ensuring stakeholder commitment to feasible and mutually satisfactory system solutions.

FIGURE 2.7

A typical spiral model



Spiral Model Phases

Spiral Model Phases	Activities performed during phase
Planning	• It includes estimating the cost, schedule and resources for the iteration. It also involves understanding the system requirements for continuous communication between the system analyst and the customer
Risk Analysis	• Identification of potential risk is done while risk mitigation strategy is planned and finalized
Engineering	• It includes testing, coding and deploying software at the customer site
Evaluation	• Evaluation of software by the customer. Also, includes identifying and monitoring risks such as schedule slippage and cost overrun

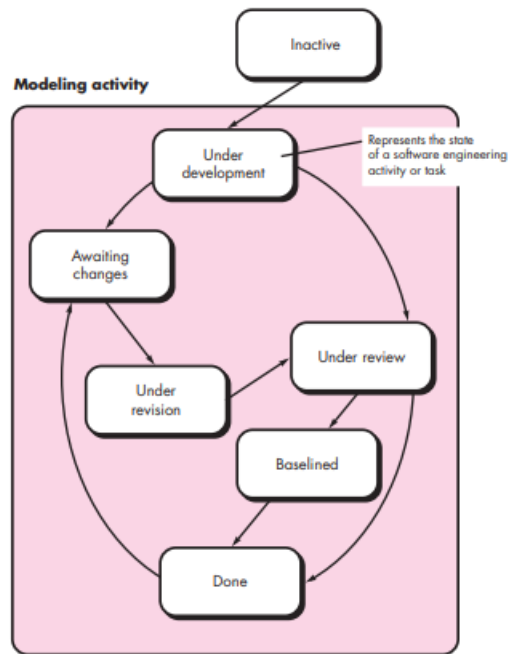
- The spiral model is a realistic approach to the development of large-scale systems and software. Because software evolves as the process progresses, the developer and customer better understand and react to risks at each evolutionary level. The spiral model uses prototyping as a risk reduction mechanism but, more important, enables you to apply the prototyping approach at any stage in the evolution of the product.
- It maintains the systematic stepwise approach suggested by the classic life cycle but incorporates it into an iterative framework that more realistically reflects the real world. The spiral model demands a direct consideration of technical risks at all stages of the project and, if properly applied, should reduce risks before they become problematic.
- But like other paradigms, the spiral model is not a panacea. It may be difficult to convince customers (particularly in contract situations) that the evolutionary approach is controllable. It demands considerable risk assessment expertise and relies on this expertise for success. If a major risk is not uncovered and managed, problems will undoubtedly occur.

Concurrent Models:

- The concurrent development model, sometimes called concurrent engineering, allows a software team to represent iterative and concurrent elements of any of the process models.
- Figure 2.8 provides a schematic representation of one software engineering activity within the modeling activity using a concurrent modeling approach. The activity modeling may be in any one of the states noted at any given time.
- Software engineering activities exist concurrently but reside in different states

FIGURE 2.8

One element of the concurrent process model



- Figure 2.8 provides a schematic representation of one software engineering activity within the modeling activity using a concurrent modeling approach. The activity—modeling—may be in any one of the states12 noted at any given time.
- Similarly, other activities, actions, or tasks (e.g., communication or construction) can be represented in an analogous manner. All software engineering activities exist concurrently but reside in different states.
- For example, early in a project the communication activity (not shown in the figure) has completed its first iteration and exists in the awaiting changes state. The modeling activity (which existed in the inactive state while initial communication was completed, now makes a transition into the under development state. If, however, the customer indicates that changes in requirements must be made, the modeling activity moves from the under development state into the awaiting changes state.

A Final Word on Evolutionary Processes:

- modern computer software is characterized by continual change, by very tight time lines, and by an emphatic need for customer–user satisfaction. In many cases, time-to-

market is the most important management requirement. If a market window is missed, the software project itself may be meaningless

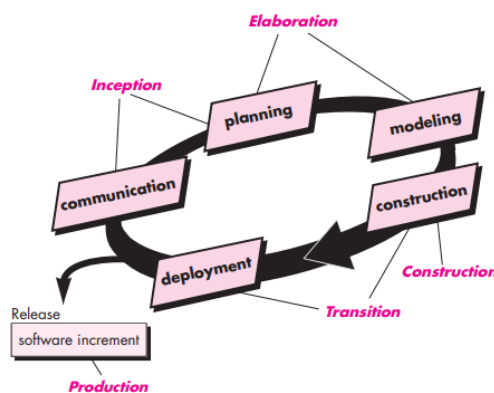
- Evolutionary process models were conceived to address these issues, and yet, as a general class of process models, they too have weaknesses. These are summarized by Nogueira and his colleagues [Nog00] :
- Despite the unquestionable benefits of evolutionary software processes, we have some concerns.
- The first concern is that prototyping [and other more sophisticated evolutionary processes] poses a problem to project planning because of the uncertain number of cycles required to construct the product. Most project management and estimation techniques are based on linear layouts of activities, so they do not fit completely.
- Second, evolutionary software processes do not establish the maximum speed of the evolution. If the evolutions occur too fast, without a period of relaxation, it is certain that the process will fall into chaos. On the other hand if the speed is too slow then productivity could be affected . . .
- Third, software processes should be focused on flexibility and extensibility rather than on high quality. This assertion sounds scary. However, we should prioritize the speed of the development over zero defects. Extending the development in order to reach high quality could result in a late delivery of the product, when the opportunity niche has disappeared. This paradigm shift is imposed by the competition on the edge of chaos.

UNIFIED Process:

- The Unified Process recognizes the importance of customer communication and streamlined methods for describing the customer's view of a system.
- "Unified method" that would combine the best features of each of their individual object-oriented analysis and design methods. The result was UML—a unified modeling language that contains a robust notation for the modeling and development of object.

FIGURE 2.9

The Unified Process



Phases of the Unified Process :

- The Inception phase of the UP encompasses both customer communication and planning activities. By collaborating with stakeholders, business requirements for the software are identified; a rough architecture for the system is proposed and a plan for the iterative, incremental nature of the ensuing project is developed.
- Architecture at this point is nothing more than a tentative outline of major subsystems and the function and features that populate them. Later, the architecture will be refined and expanded into a set of models that will represent different views of the system.
- The Elaboration phase encompasses the communication and modeling activities of the generic process model (Figure 2.9). Elaboration refines and expands the preliminary use cases that were developed as part of the inception phase and expands the architectural representation. The architectural baseline demonstrates the viability of the architecture but does not provide all features and functions required to use the system.
- The Construction phase of the UP is identical to the construction activity defined for the generic software process. Using the architectural model as input, the construction phase develops or acquires the software components that will make each use case operational for end users. To accomplish this, requirements and design models that were started during the elaboration phase are completed to reflect the final version of the software increment.
- The Transition phase of the UP encompasses the latter stages of the generic construction activity and the first part of the generic deployment (delivery and feedback) activity. Software is given to end users for beta testing and user feedback reports both defects and necessary changes. In addition, the software team creates the necessary support information (e.g., user manuals, troubleshooting guides, installation procedures) that is required for the release.
- The Production phase of the UP coincides with the deployment activity of the generic process. During this phase, the ongoing use of the software is monitored, support for the operating environment (infrastructure) is provided, and defect reports and requests for changes are submitted and evaluated.

Agile Software Development:

- Agile software engineering combines a philosophy and a set of development guidelines. The philosophy encourages customer satisfaction and early incremental delivery of software; small, highly motivated project teams, informal methods, minimal software engineering work products and overall development simplicity.
- An agile team fosters communication and collaboration among all who serve on it. The modern business environment that spawns computer-based systems and software products is fast-paced and ever changing.

Agile Process

- Any agile software process is characterized in a manner that addresses a number of key assumptions about the majority of software projects.
 1. It is difficult to predict in advance which software requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as the project proceeds.
 2. For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.
 3. Analysis, design, construction, and testing are not as predictable (from a planning point of view) as we might like. When this heavyweight, plan-driven development approach is applied to small and medium-sized business systems, the overhead involved is so large that it dominates the software development process.
- More time is spent on how the system should be developed than on program development and testing. As the system requirements change, rework is essential and, in principle at least, the specification and design has to change with the program.
- Dissatisfaction with these heavyweight approaches to software engineering led a number of software developers in the 1990s to propose new 'Agile Methods'. These allowed the development team to focus on the software itself rather than on its design and documentation.
- Agile methods universally rely on an incremental approach to software specification, development, and delivery. They are best suited to application development where the system requirements usually change rapidly during the development process. They are intended to deliver working software quickly to customers, who can then propose new and changed requirements to be included in later iterations of the system. Although these agile methods are all based around the notion of incremental development and delivery, they propose different processes to achieve this.
- Two of the most widely used methods: Extreme programming & Scrum.
- Agile methods have been very successful for some types of system development:
 1. Product development where a software company is developing a small or medium-sized product for sale.
 2. Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.
- Principles of Agile Methods :-

1. Customer Involvement :- Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.

2. Incremental delivery :- The software is developed in increments with the customer specifying the requirements to be included in each increment.

3. People not process :- The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

4. Embrace change :- Expect the system requirements to change and so design the system to accommodate these changes.

5. Maintain simplicity :- Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

- In practice, the principles underlying agile methods are sometimes difficult to realize:

1. Although the idea of customer involvement in the development process is an attractive one, its success depends on having a customer who is willing and able to spend time with the development team and who can represent all system stakeholders. Frequently, the customer representatives are subject to other pressures and cannot take full part in the software development.

2. Individual team members may not have suitable personalities for the intense involvement that is typical of agile methods, and therefore not interact well with other team members.

3. Prioritizing changes can be extremely difficult, especially in systems for which there are many stakeholders. Typically, each stakeholder gives different priorities to different changes.

4. Maintaining simplicity requires extra work. Under pressure from delivery schedules, the team members may not have time to carry out desirable system simplifications.

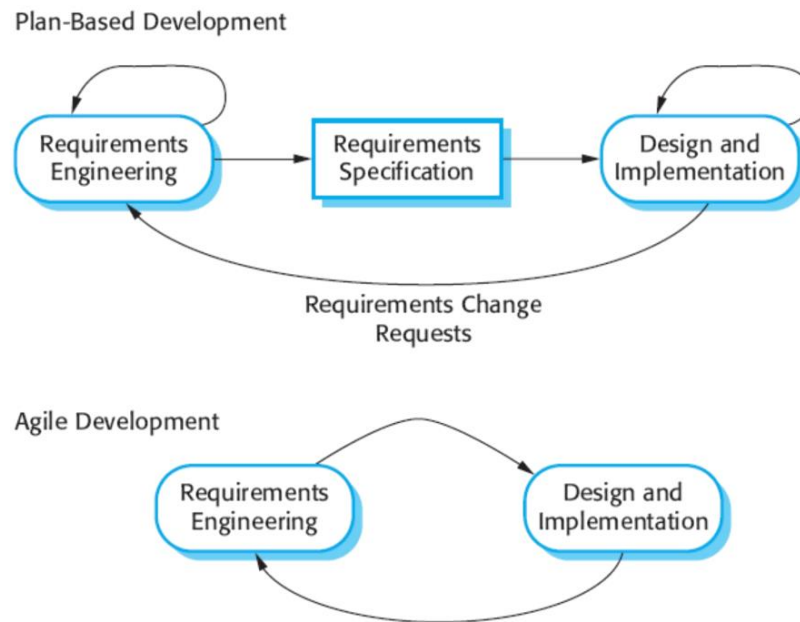
5. Many organizations, especially large companies, have spent years changing their culture so that processes are defined and followed. It is difficult for them to move to a working model in which processes are informal and defined by development teams.

- Plan Driven and Agile Development:

By contrast, a plan-driven approach to software engineering identifies separate stages in the software process with outputs associated with each stage. The outputs from one stage are used as a basis for planning the following process activity.

Figure 3.2 shows the distinctions between plan-driven and agile approaches to system specification.

Requirement Engineering = What your program should do ?
Requirement Specification = How you plan to do it ?



In a plan-driven approach, iteration occurs within activities with formal documents used to communicate between stages of the process. For example, the requirements will evolve and, ultimately, a requirements specification will be produced.

This is then an input to the design and implementation process. In an agile approach, iteration occurs across activities. Therefore, the requirements and the design are developed together, rather than separately.

A plan-driven software process can support incremental development and delivery. It is perfectly feasible to allocate requirements and plan the design and development phase as a series of increments. An agile process is not inevitably code-focused and it may produce some design documentation.

To decide on the balance between a plan-based and an agile approach, you have to answer a range of technical, human, and organizational questions:

1. Is it important to have a very detailed specification and design before moving to implementation? If so, you probably need to use a plan-driven approach.

2. Is an incremental delivery strategy, where you deliver the software to customers and get rapid feedback from them, realistic? If so, consider using agile methods.

3. How large is the system that is being developed? Agile methods are most effective when the system can be developed with a small co-located team who can communicate informally. This may not be possible for large systems that require larger development teams so a plan-driven approach may have to be used.

4. What type of system is being developed? Systems that require a lot of analysis before implementation (e.g., real-time system with complex timing requirements) usually need a fairly detailed design to carry out this analysis. A plan-driven approach may be best in those circumstances.

5. What is the expected system lifetime? Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team. However, supporters of agile methods rightly argue that documentation is frequently not kept up to date and it is not of much use for long-term system maintenance.